

∴ {cover}

RHEL 9 · RHCSA 实操讲义

13

系统日志、Journal、

rsyslog 与日志轮转

从“事件发生”到“证据可查”：把范围、字段、路由、持久化与轮转放进同一条证据链。

对象模型操作语义验证诊断经典任务

大字号阅读版

⋮

⋮ {navigation}

本章阅读导航

先抓住一条主线：日志不是某一个文件，而是事件从产生、收集、筛选、路由、保存到轮转的完整证据链。每次查询或变更，都要先回答“我正在证明哪一层”。

∴ { .model-grid } ∴ { .model-step } 01

识别事件来源

内核、unit、syslog 客户端，还是应用自有文件？ ∴ ∴ { .model-step } 02

限定 journal 范围

先定 boot 与时间，再定 unit、tag、PID 或字段。 ∴ ∴ { .model-step } 03

解释结构化字段

区分消息正文、客户端字段和 journal 可信字段。 ∴ ∴ { .model-step } 04

验证路由规则

用唯一事件证明 selector 命中并完成 action。 ∴ ∴ { .model-step } 05

判断持久与保留

当前可读、跨 boot 存在、尚未被清理是不同状态。 ∴ ∴ { .model-step } 06

验证文本轮转

先 dry-run，再在受控对象上轮转并验证新写入。 ∴ ∴

专题地图 - **知识专题** 从事件产生到可审计证据 - **操作专题** 按范围和字段收缩 journal - **操作专题** 解释结构化字段与输出格式 - **操作专题** 建立持久 journal 与保留预算 - **操作专题** 用 `logger` 构造验证事件 - **操作专题** 配置 rsyslog 本地路由 - **操作专题** 按层验证远端转发 - **操作专题** 管理 logrotate 状态与策略 - **诊断专题** 日志查不到、写不到、转不到和轮不动 - **经典任务** facility 路由与轮转；跨 boot 持久性

阅读时持续回答 1. 目标日志可能存在于哪个范围？ 2. 当前查询限定了哪个 boot 和时间窗口？ 3. 哪个字段能够唯一缩小对象？ 4. 这条证据来自客户端，还是 journal 补充？ 5. selector 命中后执行了什么 action？ 6. 当前可读是否等于重启后仍可读？ 7. logrotate 为什么认为“本次不轮转”？ 8. 操作结果能证明什么，又不能证明什么？ ∴ { .note-box } 本章使用 `systemctl` 观察和应用日志服务变更，但不重新展开 unit 状态模型；时间戳异常只建立排查接口，时间同步完整机制留给第 14 章。 ∴

∴

∴ { .chapter-opening }

第 13 章 · 正文

系统日志不是一个文件，而是一条证据链

一台 RHEL 9 主机上的日志可能同时出现在结构化 journal、传统 `/var/log` 文本文件、远端日志主机以及应用自己的目录中。`systemd-journald`、`rsyslog` 和 `logrotate` 都与“日志”有关，却分别管理不同对象：`journald` 收集并索引事件，`rsyslog` 根据规则执行写文件或转发，`logrotate` 管理普通文本文件的生命周期。

最常见的误判来自对象混淆。`rsyslog.service` 处于 active，只能证明进程当前运行，不能证明某条 selector 命中过；`journalctl -b -1` 没有输出，不等于上一次启动没有故障；出现 `.1` 或 `.gz`，也不等于写入进程已经切换到新的活动文件。

本章按“对象与范围 → 查询与字段 → 配置与路由 → 持久和保留 → 轮转与诊断”推进。第 12 章已经建立 `systemd unit` 的状态与控制入口；第 14 章将完整讲解时间、时区和 `chrony`；应用框架自己的日志库不在本章展开。

事件产生者

- `journald` 收集、附加字段并存储
- `journalctl` 按 boot、时间、对象和字段检索
- `rsyslog selector` 选择消息并执行 action
- 本地文件或远端接收端留下结果
- `logrotate` 管理文本文件的轮转和保留
- 用新事件、跨 boot 与轮转后写入完成验收

...

... {`.concepts`}

核心概念

... {`.concept-card`} 概念 **Journal** 是由 `systemd-journald` 维护的结构化事件集合，而不是一份普通文本文件。每条 entry 除了 `MESSAGE`，还可以包含 boot、进程、用户、unit、可执行文件和优先级等字段，因此能够按系统对象精确查询。journal 当前可查并不自动意味着它会跨重启保留；存储位置与保留策略必须另行确认。

...

... {`.concept-card`} 概念 **结构化字段** 把“看到一行文字”变成“识别一条事件”。`SYSLOG_IDENTIFIER`、`PRIORITY` 等字段可能由客户端提供；以 `_` 开头的 `_PID`、`_UID`、`_SYSTEMD_UNIT`、`_BOOT_ID` 等通常由 journal 在接收时补充，证据强度更高。字段不存在不等于事件无效，只说明不能用该维度筛选。...

... {`.concept-card`} 概念 **Boot 范围** 是 journal 查询的第一层时间边界。`-b` 选择当前启动，`-b -1` 选择上一次仍可查询的启动；它们依赖 journal 中实际保留的 boot。只有当前 boot 可见时，应继续区分“从未持久保存”“历史已被清理”“权限不足”，而不是把空结果解释成没有事件。...

... {`.concept-card`} 概念 **Facility 与 severity** 分别描述 syslog 消息的类别和严重程度。传统 `rsyslog selector` 以 `facility.priority` 选择消息；`local0.notice` 表示 notice 以及更严重级别，而 `local0.=notice` 只精确匹配 notice。severity 数字越小越严重，这与“数字越大越严重”的直觉相反。...

⋮ {concept-card} 概念 日志路由 是“选择条件 + 动作”的组合。selector 命中后，action 可以写入本地文件、转发到远端或进入其他处理链。配置语法通过只证明文件可解析；只有带唯一标识的新测试事件真正出现在目标位置，才能证明规则已加载、已命中且 action 已完成。⋮

⋮ {concept-card} 概念 持久与保留 是两个不同问题。持久性回答“重启后能否继续读取”，保留回答“在容量、时间和清理操作作用下能保存多久”。`Storage=persistent` 建立持久存储方向，但 `SystemMaxUse=`、`MaxRetentionSec=` 和 `vacuum` 仍会决定历史证据何时被回收。⋮

⋮ {concept-card} 概念 logrotate 状态 记录普通文本日志上一次轮转的时间，并与策略中的 `daily`、`weekly`、`size` 等条件共同决定本次是否轮转。文件很大不代表一定到期，配置正确也不代表调度入口已经执行。`-d` 用于解释决策且不改文件，`-f` 会真实强制轮转，二者风险完全不同。⋮ ⋮

⋮ {quickref}

操作语义速查

先建立接口地图，再进入正文。参数按“范围、对象、优先级、输出”纵向组织，避免把多个选项挤成一行。

⋮ {quick-command}

journalctl

SYNOPSIS

```
journalctl [OPTIONS...] [MATCHES...]
```

检索 journal entry。稳定顺序是先确定 boot 与时间范围，再按 unit、tag、PID 或字段选择对象，最后决定优先级、输出格式和条数。

重要参数 / 形式

`--list-boots`

列出当前仍可查询的启动轮次、boot ID 和时间范围。

`-b [ID|OFFSET]`

选择一次启动；`-b` 为当前 boot，`-b -1` 为上一次仍被保留的 boot。

`-u UNIT` / `-t TAG`

分别按 systemd unit 和 `SYSLOG_IDENTIFIER` 选择来源。

`--since TIME` / `--until TIME`

限定时间窗口；查询为空时先放宽最窄条件并核对当前时间与时区。

`-p LEVEL[...LEVEL]`

按 severity 过滤；单个级别包含该级别及更严重记录。

`FIELD=value`

使用结构化字段匹配；不同字段通常为 AND，同一字段多个值通常为 OR。

`-o FORMAT`

选择输出格式；`verbose` 适合发现字段，`json-pretty` 适合机器处理，`cat` 只显示正文。

...

::: {.quick-command}

logger

SYNOPSIS

```
logger [OPTIONS...] [MESSAGE]
```

向 syslog/journal 链生成可控测试事件。验证配置时应使用唯一 tag 和唯一正文，避免把旧消息误当作新规则的结果。

重要参数 / 形式

-p facility.level

设置 facility 与 severity，例如 `local0.notice`；它不直接指定目标文件。

-t TAG

设置 `SYSLLOG_IDENTIFIER`，便于 `journalctl -t TAG` 查询。

-i

在消息中记录 logger 进程 ID；用于演示 PID 字段，不应把短命 PID 当长期身份。

-f FILE

将文件内容逐行写入日志链；大文件和敏感内容使用前必须确认影响。

...

::: {.quick-command}

journald 配置

SYNOPSIS

```
# /etc/systemd/journald.conf.d/60-local.conf
[Journal]
Storage=persistent
SystemMaxUse=512M
MaxRetentionSec=30day
```

通过主配置或 drop-in 改变 journal 的存储与保留策略。管理员配置优先放入 `/etc/systemd/journald.conf.d/*.conf`，避免直接修改 vendor 文件。

重要参数 / 形式

Storage=persistent

使用 `/var/log/journal` 保存可跨重启读取的 journal。

Storage=volatile

使用 `/run/log/journal`，重启后内容消失。

```
SystemMaxUse= / RuntimeMaxUse=
```

分别限制持久和运行时 journal 的最大占用。

```
MaxRetentionSec=
```

限制 journal 文件的最长保留时间；仍受磁盘与容量策略影响。

```
journalctl --flush
```

在持久存储可用时，把运行时 journal 刷入持久区域；它不能恢复已经丢失的历史。

:::

::: {quick-command}

rsyslog selector / action

SYNOPSIS

```
facility.priority    action
```

传统规则用 selector 选择消息，再把它交给文件或远端 action。本章以传统语法建立 RHCSA 主线，复杂 RainerScript 只用于说明工作边界。

重要参数 / 形式

```
local0.notice /var/log/exam-audit.log
```

匹配 local0 的 notice 及更严重级别，并写入本地文件。

```
local0.=notice /var/log/exam-notice.log
```

只精确匹配 notice。

```
*.info;mail.none /var/log/messages
```

组合 selector，并使用 none 排除某个 facility。

```
*.* @loghost:514
```

使用 UDP 转发。

```
*.* @@loghost:514
```

使用 TCP 转发；连接建立不等于接收端已经落盘。

:::

::: {quick-command}

```
rsyslogd -N1
```

SYNOPSIS

```
rsyslogd -N1
```

在应用配置前执行静态解析检查。它适合发现语法错误、未知模块或配置读取问题，但不能证明 selector 会命中，也不能证明目标路径、网络或远端 action 可用。

重要参数 / 形式

-N1

检查配置语法和基本有效性后退出，不启动第二个常驻 rsyslogd。

systemctl restart rsyslog.service

在静态检查通过后让服务重新读取配置；unit 状态模型归第 12 章。

journalctl -u rsyslog.service -b

查看应用配置后的服务日志；仍需以目标层新事件完成最终验证。

...

... { .quick-command }

logrotate

SYNOPSIS

```
logrotate [OPTIONS...] <config-file>
```

读取轮转策略和状态文件，判断普通文本日志是否到期，并执行重命名、压缩、创建新文件和脚本钩子。先解释决策，再在明确测试对象上执行真实轮转。

重要参数 / 形式

-d

debug/dry-run；解释读取到的策略和轮转判断，不修改文件或状态。

-f

强制真实轮转，即使状态文件认为尚未到期；不得对未知生产对象直接使用。

-s STATEFILE

使用独立状态文件，便于隔离受控测试。

rotate N / compress

保留 N 份归档，并压缩旧文件。

size / daily / weekly

分别按大小或时间周期建立触发条件。

create MODE OWNER GROUP

轮转后创建新的活动文件并设置权限与属主。

postrotate ... endscript

轮转后通知写入进程重新打开文件；具体信号或命令必须匹配实际程序。

... :::

... { #RHCSA-13-K01 .topic .knowledge data-kind="knowledge-topic" }

[知识专题] 从事件产生到可审计证据：先认清对象与状态

日志排错的第一步不是打开 `/var/log/messages`，而是判断事件可能经过哪条路径。相同事件可以同时存在于 journal 和文本文件；应用也可能完全绕开 syslog，直接写自己的日志。只有先画出路径，后续查询才有明确边界。

① [知识点] journald 把多种输入统一为 entry

`systemd-journald` 可以接收内核消息、启动阶段输出、systemd 服务的标准输出与标准错误，以及经 syslog 接口提交的消息。它为每条 entry 保存正文和元数据，使管理员能够按 boot、unit、进程、用户、主机和优先级查询。

journal 是查询对象，不等于“所有应用日志的全集”。若应用只写 `/opt/app/log/app.log`，journal 中可能没有对应正文。反过来，unit 的标准错误可能只存在于 journal，而没有被任何 rsyslog 规则写入文本文件。

② [知识点] rsyslog 处理的是规则链，不是“第二份 journal”

rsyslog 的基础模型是：

```
input
→ selector / filter
→ action
```

在典型 RHEL 9 主机中，它接收 syslog 消息，并根据 facility、severity 或属性执行 action。action 可以是本地文件、远端转发或其他处理。它不会替代 journal 的字段索引，也不会自动管理目标文件的长期轮转。

③ [知识点] 文本日志和 journal 拥有独立生命周期

journal 文件由 journald 自己维护，容量通过 journald 配置和 `journalctl --vacuum=*` 控制。普通文本文件由写入进程持续追加，再由 logrotate 按策略轮转。两套机制不能互相替代：

```
journalctl --vacuum-size=500M
```

不会轮转 `/var/log/exam-audit.log`；而：

```
logrotate /etc/logrotate.conf
```

也不会删除二进制 journal entry。

④ [知识点] “当前可读”“跨 boot 存在”“仍被保留”必须分别判断

- 当前可读：当前用户能在现有 journal 文件中查询到 entry。
- 跨 boot 存在：重启后仍保留前一 boot 的 journal。
- 仍被保留：容量、时间、磁盘压力或 vacuum 尚未删除它。

- 目标可读：当前用户具备读取该 journal 或文本文件的权限。

因此，`journalctl -b -1` 没有结果时，至少有四种假设，而不是一个结论。

⑤ [边界] 日志时间是坐标，不在本章重建时间系统

时间范围查询依赖系统时间与时区。若已知事件没有落在预期窗口，先放宽时间条件并查看当前时间；若多主机事件顺序异常，再把时间同步列为独立假设。RTC、chronyd、source、offset 和 step/slew 归第 14 章完整展开。

∴ {cheatsheet} Cheatsheet 先问事件来源，再问 journal 范围；journald 负责收集与索引，rsyslog 负责选择与 action，logrotate 负责普通文本文件生命周期；“当前可读、持久、保留、权限”四个状态不能互相替代。∴ ∴

∴ {#RHCSA-13-O01 .topic .operation data-kind="operation-topic"}

[操作专题] 先限定范围，再按对象收缩 journal

`journalctl` 参数很多，但可以稳定分成四组：范围、对象、优先级和输出。推荐顺序是“boot/时间 → unit/tag/字段 → priority → 条数与格式”。一开始就叠加所有条件，最容易制造一个无法解释的空结果。

① [查询] 用 `--list-boots` 证明有哪些启动轮次可查

```
journalctl --list-boots
journalctl -b --no-pager
journalctl -b -1 --no-pager
journalctl --boot=<BOOT_ID> --no-pager
```

`--list-boots` 是 `-b -1` 的前置证据。只有列表中存在上一 boot，`-b -1` 才有明确对象。列表只显示当前 boot，证明的是“当前可用 journal 里只有这一轮”，不能证明历史从未产生。

② [查询] 用 unit、tag 和字段表达来源

```
journalctl -u sshd.service -b --no-pager
journalctl -t exam-audit --since today --no-pager
journalctl _PID=2450 -b --no-pager
journalctl _SYSTEMD_UNIT=sshd.service _UID=0 \
  --since '-30 min' --no-pager
```

`-u` 是按 systemd unit 的便捷匹配；`-t` 匹配 `SYSLOG_IDENTIFIER`。PID 可能在不同时间被复用，因此较长窗口必须同时限定 boot、时间或 unit。字段不存在时，查询不会自动改用其他字段。

可先枚举已有值：

```
journalctl -F _SYSTEMD_UNIT
journalctl -F SYSLOG_IDENTIFIER
```

③ [查询] 用时间窗口保留故障上下文

```
journalctl --since today --no-pager
journalctl --since '-15 min' --no-pager
journalctl --since '2026-07-12 13:00:00' \
  --until '2026-07-12 13:20:00' --no-pager
```

绝对时间适合题目给出明确事件点，相对时间适合刚完成的验证。若结果为空，先去掉 `--until` 或扩大 `--since`，再核对 boot 与时区。不要把过窄窗口产生的空结果当作“事件不存在”。

④ [查询] 认识 priority 的方向

severity 顺序是：

名称	数字	典型含义
emerg	0	系统不可用
alert	1	必须立即处理
crit	2	严重状态
err	3	错误
warning	4	警告
notice	5	正常但重要
info	6	信息
debug	7	调试

```
journalctl -p warning -b --no-pager
journalctl -p err..alert -b --no-pager
```

单个 `warning` 包含 `warning` 及更严重记录。初次调查只看 `err` 可能遗漏错误前的 `warning` 或 `notice`，应先保留上下文，再缩小优先级。

⑤ [查询] 用条数、方向和 follow 控制阅读成本

```
journalctl -u sshd.service -b -n 80 --no-pager
journalctl -u sshd.service -b -r -n 30 --no-pager
journalctl -u sshd.service -f
```

`-n` 限制条数，`-r` 倒序显示，`-f` 跟随新消息。`-f` 适合观察新测试事件，却不能证明旧历史完整；终端中断 `follow` 也不会停止被观察服务。

⑥ [验证] 为每个查询写明证据边界

```
journalctl -u rsyslog.service -b -n 20 --no-pager
```

能证明：当前 journal 中存在与 rsyslog unit 关联的最近记录。不能证明：某条自定义 selector 已命中、目标文件可写、远端已接收。

∴ {.cheatsheet} Cheatsheet --list-boots 先确定 boot；-b/-u/-t/FIELD=value 再缩小对象；时间为空先放宽；-p 数字越小越严重；每条查询都要注明“能证明什么、不能证明什么”。∴ ∴

∴ {#RHCSA-13-O02 .topic .operation data-kind="operation-topic"}

[操作专题] 用字段解释记录，而不是只读消息正文

当一条日志看似“来自某服务”时，真正需要确认的是：谁发送、属于哪个 boot、由哪个 unit 管理、以什么身份运行、优先级是什么。-o verbose 是字段发现入口，随后再选择最有区分度的字段进行匹配。

① [查询] 用 -o verbose 展开完整 entry

```
journalctl -u sshd.service -b -n 1 -o verbose
```

重点观察：

字段	用途	边界
MESSAGE	消息正文	可由客户端控制
PRIORITY	severity 数字	0 最严重，7 为 debug
SYSLOG_IDENTIFIER	客户端标识	可由客户端设置
_BOOT_ID	启动身份	跨 boot 的稳定范围字段
_PID / _UID / _GID	发送进程和身份	由 journal 接收侧补充
_COMM / _EXE	进程名与可执行文件	路径可能因升级变化
_SYSTEMD_UNIT	所属 systemd unit	并非所有 entry 都有
_HOSTNAME	记录中的主机名	主机名变更要结合时间

② [知识点] 下划线字段通常比客户端字段更可信

普通字段可以由客户端提交；以下划线开头的 trusted fields 通常由 journal 根据接收时环境生成，客户端不能任意伪造。因此调查来源时，优先组合 _BOOT_ID、_PID、_UID、_EXE 和 _SYSTEMD_UNIT，而不是只相信正文里写着“来自 sshd”。

可信不等于永远存在。内核消息、早期启动消息或脱离 unit 的进程可能缺少 `_SYSTEMD_UNIT`；此时应换用 `_TRANSPORT`、`_COMM`、`_EXE` 或 `boot` 范围继续调查。

③ [查询] 理解字段表达式的 AND 与 OR

```
journalctl _SYSTEMD_UNIT=sshd.service _UID=0 -b
```

不同字段之间是 AND：同时满足 unit 与 UID。

```
journalctl _SYSTEMD_UNIT=sshd.service \
           _SYSTEMD_UNIT=chronyd.service -b
```

同一字段多个值通常为 OR：匹配任一 unit。

显式 `+` 可以连接两组完整表达式：

```
journalctl _SYSTEMD_UNIT=sshd.service + \
           SYSLOG_IDENTIFIER=exam-audit
```

复杂表达式应先分别运行每个子条件，确认都有结果，再合并。否则一个拼写错误就会把整组查询变成空集。

④ [查询] 按阅读和机器处理目的选择输出格式

```
journalctl -u sshd.service -b -n 20 -o short-iso-precise
journalctl -t exam-audit -n 5 -o cat
journalctl -t exam-audit -n 2 -o json-pretty
```

- `short-iso-precise`：便于人工比较精确时间。
- `cat`：只显示正文，适合脚本读取单一消息，但会丢失字段上下文。
- `json-pretty`：保留结构供机器处理。
- `verbose`：发现字段，不适合作为日常大量输出。

改变 `-o` 只改变呈现，不会改变 entry 或其保存状态。

⑤ [验证] 输出中出现字段，不等于业务身份已经成立

`_SYSTEMD_UNIT=web.service` 证明 journal 把该进程归属于该 unit；它不证明 Web 端口正在监听、请求成功或内容正确。unit 功能验收仍要回到第 12 章建立的服务证据链。

⋮ {cheatsheet} Cheatsheet 先用 `-o verbose` 发现字段；来源判断优先 trusted fields；不同字段 AND、同字段多值 OR；`-o cat` 会丢字段；字段归属不能替代业务功能验证。⋮ ⋮

⋮ {#RHCSA-13-O03 .topic .operation data-kind="operation-topic"}

[操作专题] 建立持久 journal，并控制占用与保留

持久化配置的目标不是“创建一个目录”，而是让有效配置、存储位置、当前 entry、跨 boot entry 和保留预算形成闭环。若当前维护窗口不允许安全重启，只能完成配置核对和重启前证据，不能声称跨重启已经成功。

① [查询] 先调查有效配置与当前存储

```
journalctl --list-boots
journalctl --disk-usage
ls -ld /run/log/journal /var/log/journal 2>/dev/null
systemd-analyze cat-config systemd/journald.conf
```

若目标版本不支持 `systemd-analyze cat-config` 的该用法，则直接检查：

```
grep -R -nE '^[[:space:]]*(Storage|SystemMaxUse|RuntimeMaxUse|MaxRetentionSec)= ' \
/etc/systemd/journald.conf \
/etc/systemd/journald.conf.d \
/usr/lib/systemd/journald.conf.d 2>/dev/null
```

调查要回答：有效 `Storage=` 是什么、是否存在管理员 `drop-in`、当前有几个 `boot`、`journal` 占用多大。

② [操作] 用管理员 `drop-in` 表达目标策略

```
# /etc/systemd/journald.conf.d/60-persistent.conf
[Journal]
Storage=persistent
SystemMaxUse=512M
MaxRetentionSec=30day
```

- `Storage=persistent`：把长期 `journal` 放入 `/var/log/journal`。
- `SystemMaxUse=512M`：限制持久 `journal` 使用量。
- `MaxRetentionSec=30day`：限制最长保留时间。

这些值是任务参数，不是推荐所有主机统一采用的默认值。生产环境应依据磁盘预算、故障回溯和审计要求确定。

③ [操作] 让运行中的 `journald` 读取配置

```
systemctl restart systemd-journald.service
journalctl --flush
journalctl --disk-usage
```

`restart` 和 `flush` 的对象不同：`restart` 让服务重新读取配置；`flush` 在持久存储可用时把运行时数据移入持久区域。两条命令成功仍不能证明“下一次重启后可查”，只能为跨 `boot` 验证建立条件。

④ [验证] 用唯一事件建立跨 boot 证据

```
TEST_ID="rhcsa13-persist-$(date +%s)"
logger -p local0.notice -t rhcsa13-persist "$TEST_ID"

journalctl -t rhcsa13-persist --since '-2 min' --no-pager
journalctl --list-boots
```

当前能证明：新事件已进入当前 journal，tag 和正文可查询。下一次安全重启后必须再执行：

```
journalctl --list-boots
journalctl -b -1 -t rhcsa13-persist --no-pager
```

只有在上一 boot 找到重启前 TEST_ID，才能证明跨重启持久。

⑤ [查询] 区分占用观察和清理动作

```
journalctl --disk-usage
journalctl --rotate
journalctl --vacuum-time=30d
journalctl --vacuum-size=500M
journalctl --vacuum-files=20
```

`--disk-usage` 只观察。`vacuum` 会删除归档 journal，是不可逆的证据清理动作。若确需清理，先确认审计与故障调查要求；必要时先 `rotate`，使活动文件归档后再按目标条件 `vacuum`。

⑥ [安全边界] 磁盘紧张时不要把 vacuum 当作第一反应

正确顺序是：

```
确认 journal 占用
→ 判断是否存在异常高频消息
→ 确认业务和审计保留要求
→ 调整长期预算或修复消息源
→ 只删除允许删除的归档证据
→ 再次检查占用和日志速率
```

直接清空日志会破坏事故现场，也可能让根因继续高速产生日志。

∴ {cheatsheet} Cheatsheet 有效配置 + 存储目录 + 当前 TEST_ID + 跨 boot TEST_ID 才是持久闭环；`--disk-usage` 是观察，`vacuum` 是删除；新配置不能恢复已经丢失的历史。∴ ∴

∴ {#RHCSA-13-O04 .topic .operation data-kind="operation-topic"}

[操作专题] 用 `logger` 构造可重复、可判定的验证事件

验证消息必须能与历史消息区分。固定正文如“test”可能早已存在，既不能证明新规则已经加载，也不能确定它来自本次操作。最小模式是“唯一 TEST_ID + 明确 facility/severity + 明确 tag”。

① [操作] 生成具有唯一身份的测试事件

```
TEST_ID="rhcsa13-$(date +%s)"
logger -p local0.notice -t exam-audit "$TEST_ID"
```

这里：

- `local0` 是 facility;
- `notice` 是 severity;
- `exam-audit` 成为 `SYSLOG_IDENTIFIER`;
- `$TEST_ID` 是本次测试唯一正文。

② [验证] 先证明事件进入 journal

```
journalctl -t exam-audit --since '-2 min' --no-pager
journalctl SYSLOG_IDENTIFIER=exam-audit MESSAGE="$TEST_ID" \
  --since '-2 min' --no-pager
```

第一条适合人工查看，第二条把 tag 和正文组合起来。若 journal 没有事件，先调查 logger 调用、当前时间、权限和 journald；不要直接跳到 rsyslog 文件层。

③ [验证] 再证明目标 action 完成

```
grep -F "$TEST_ID" /var/log/exam-audit.log
```

journal 有而文件无，说明事件产生和 journald 收集基本成立，故障更可能位于 rsyslog 配置加载、selector、目标路径、权限或 SELinux 层。此时重新发送新的 TEST_ID，而不是复用旧消息。

④ [边界] `logger -p` 不指定目标文件

`-p local0.notice` 只描述消息类别和级别。消息写入哪个文件、是否远端转发，由 rsyslog 规则决定。把 `logger -p` 理解成“写文件选项”会混淆 producer 与 router。

⑤ [安全边界] 不把敏感内容当测试正文

日志可能被写入多个本地文件并转发到集中平台。口令、私钥、token、Vault 明文和个人数据不应作为验证消息。生产环境还应考虑日志采样、脱敏和保留政策，但应用日志框架不在本章展开。

⋮ {.cheatsheet} Cheatsheet 每次变更后创建新 TEST_ID；先在 journal 找，再到目标 action 找；`logger -p` 设置消息属性，不指定目标文件；测试正文不能含秘密。⋮ ⋮

⋮ {#RHCSA-13-O05 .topic .operation data-kind="operation-topic"}

[操作专题] 配置 rsyslog selector 与本地文件 action

本地路由任务的核心不是写出一行配置，而是证明四件事：配置可解析、服务读取了新规则、测试消息满足 selector、文件 action 成功写入。每一层都使用不同证据。

① [知识点] `facility.priority` 默认是阈值匹配

```
local0.notice    /var/log/exam-audit.log
```

它匹配 `local0.notice`、`local0.warning`、`local0.err` 等 notice 及更严重消息，不匹配 info 和 debug。

```
local0.=notice   /var/log/exam-notice.log
```

只精确匹配 notice。若题目要求“notice 及以上”，使用 `.=notice` 会漏掉更严重事件。

② [操作] 在独立片段中写入最小规则

```
# /etc/rsyslog.d/60-exam-audit.conf
local0.notice    /var/log/exam-audit.log
```

变更前先查冲突：

```
grep -R -n 'local0\|exam-audit' \
/etc/rsyslog.conf /etc/rsyslog.d 2>/dev/null
```

同一消息可以命中多条规则。发现已有规则时，应判断是否允许多目标写入，而不是直接删除系统配置。

③ [验证] 静态检查只能通过第一道门

```
rsyslogd -N1
```

能证明：配置文件和引用内容可被 rsyslogd 解析。不能证明：服务已重新读取、selector 会命中、目录可写、SELinux 允许写入、文件内容正确。

静态检查通过后应用：

```
systemctl restart rsyslog.service
systemctl is-active rsyslog.service
journalctl -u rsyslog.service -b -n 30 --no-pager
```

④ [验证] 使用新事件完成目标层验收

```
TEST_ID="rhcsa13-local-$(date +%s)"
logger -p local0.notice -t exam-audit "$TEST_ID"

journalctl -t exam-audit --since '-2 min' --no-pager
grep -F "$TEST_ID" /var/log/exam-audit.log
stat -c '%U %G %a %n' /var/log/exam-audit.log
```

目标文件中的 TEST_ID 证明 selector 与 file action 对这条消息成立；stat 再检查文件身份与模式。它仍不证明 logrotate 策略正确。

⑤ [诊断] journal 有消息、文件没有时选择下一条证据

```
journal 中 TEST_ID 存在
→ rsyslogd -N1
→ 检查 include 与规则顺序
→ 查看 rsyslog unit 日志
→ 检查目标目录和文件权限
→ 检查 AVC/SELinux 证据
→ 最小修复
→ 生成新的 TEST_ID 再验证
```

不要用 chmod 777 或关闭 SELinux 绕过调查。SELinux 完整规则归第 28、29 章，本章只把 AVC 作为日志写入失败的一条证据。

::: {.cheatsheet} Cheatsheet facility.priority 是阈值，facility.=priority 是精确；-N1 只证明可解析；最终必须以新 TEST_ID 进入目标文件验收。::: :::

::: {#RHCSA-13-O06 .topic .operation data-kind="operation-topic"}

[操作专题] 配置远端转发，并按层证明消息真正到达

远端日志链至少跨越发送规则、名称解析、网络、协议监听、接收规则和落盘六层。客户端建立连接只是中间证据，不能替代接收端的最终查询。

① [操作] 使用传统转发 action 表达协议

```
# UDP
*.* @loghost.example.com:514

# TCP
*.* @@loghost.example.com:514
```

单个 @ 表示 UDP，双 @@ 表示 TCP。TCP 提供连接和重传基础，但并不自动等于 TLS、可靠磁盘队列或端到端交付保证。

生产系统更常使用显式 action：

```
action(
  type="omfwd"
  target="loghost.example.com"
  port="514"
  protocol="tcp"
  queue.type="linkedList"
)
```

本章不展开 TLS 证书、复杂队列和集中平台架构，只强调它们不是基础 TCP 配置自动获得的能力。

② [查询] 在客户端分层确认发送前提

```
rsyslogd -N1
getent hosts loghost.example.com
ss -tnp | grep ':514'
journalctl -u rsyslog.service -b -n 50 --no-pager
```

- 名称解析失败，应先修复目标解析，不要改 selector。
- TCP 没有连接，继续查路由、防火墙和监听。
- TCP 已连接，只证明传输层建立，不证明服务端规则命中或落盘。

③ [操作] 用唯一事件进行端到端测试

客户端：

```
TEST_ID="rhcsa13-remote-$(date +%s)"
logger -p local0.notice -t rhcsa13-remote "$TEST_ID"
```

接收端最终应按真实配置查询：

```
journalctl -t rhcsa13-remote --since '-5 min' --no-pager
grep -R -F "$TEST_ID" /var/log 2>/dev/null
```

只有接收端出现同一 TEST_ID，才能证明消息跨主机到达。若题目要求指定文件，还必须在该文件中找到它。

④ [诊断] 按“发送 → 网络 → 接收 → action”推进

```
客户端 journal 有 TEST_ID
→ 客户端 rsyslog 配置可解析
→ 目标名称和地址正确
→ 网络与端口可达
→ 服务端监听正确协议
→ 服务端接收规则加载
→ 服务端目标文件或 journal 中出现 TEST_ID
```

每一步只验证一层。不要因为 telnet 或 TCP connect 成功，就跳过服务端日志与文件验收。

⑤ [安全边界] 不把无加密 TCP 描述成安全集中日志

远端日志可能包含账号、路径、主机结构和故障细节。跨不可信网络时，需要 TLS、证书验证、队列和访问控制。由于这些内容超出 RHCSA 本章范围，本章只标记边界，不提供“看似完整但未经验证”的生产模板。

::: {cheatsheet} Cheatsheet @ UDP, @@ TCP; 连接成功不是落盘；最终证据必须在接收端找到同一 TEST_ID；TLS 与可靠队列不是基础 TCP 自动拥有。::: :::

::: {#RHCSA-13-O07 .topic .operation data-kind="operation-topic"}

[操作专题] 用 logrotate 管理文本日志的状态、轮转与保留

logrotate 的决策来自“策略 + 状态 + 当前文件”。排错时先让 -d 解释为什么轮转或不轮转，再决定是否需要受控的真实测试。直接 -f /etc/logrotate.conf 可能同时改变大量系统日志，是典型高风险捷径。

① [知识点] 配置来源与状态文件共同参与判断

常见入口：

```
/etc/logrotate.conf
/etc/logrotate.d/*
/var/lib/logrotate/logrotate.status
```

主配置通常 include 片段。状态文件记录上次轮转时间。即使文件已经很大，daily 规则若刚执行过，仍可能不轮转；size 规则则按当前大小判断。

② [知识点] 区分时间条件、大小条件和保留动作

指令	作用
daily / weekly / monthly	按时间周期判断

指令	作用
<code>size 100M</code>	达到大小即轮转
<code>minsize 10M</code>	时间到且至少达到该大小
<code>maxsize 500M</code>	即使时间未到，超过该大小也可轮转
<code>rotate 7</code>	保留 7 份归档
<code>compress</code>	压缩旧归档
<code>delaycompress</code>	下一轮再压缩最近归档
<code>missingok</code>	文件缺失时不报致命错误
<code>notifempty</code>	空文件不轮转
<code>create 0640 root root</code>	创建新活动文件并设置身份

触发条件回答“何时轮转”，保留动作回答“轮转后怎样处理”。两类指令不能混为一谈。

③ [操作] 编写自包含策略

```
# /etc/logrotate.d/exam-audit
/var/log/exam-audit.log {
    daily
    rotate 7
    compress
    missingok
    notifempty
    create 0640 root root
    postrotate
        /usr/bin/systemctl kill -s HUP --kill-whom=main \
        rsyslog.service >/dev/null 2>&1 || true
    endscript
}
```

`postrotate` 请求 `rsyslog` 重新打开文件。真实主机应与系统自带 `rsyslog` 轮转片段核对信号和命令；不同应用的 `reopen` 方式不同，不能机械复制。

④ [查询] 先 `dry-run` 解释决策

```
logrotate -d /etc/logrotate.conf
```

重点观察：目标片段是否被 `include`、文件是否匹配、读取到哪个状态、为何到期或未到期。`debug` 不修改文件和状态，适合首次审查。

⑤ [操作] 只在受控对象上强制真实轮转

```
logrotate -s /tmp/rhcsa13-exam.status \  
-f /etc/logrotate.d/exam-audit
```

这条命令会真实改动 `/var/log/exam-audit.log`。独立状态文件隔离测试状态，却不会隔离文件本身。执行前必须确认片段只包含目标测试文件。

⑥ [验证] 轮转结果至少检查四层

```
ls -l /var/log/exam-audit.log*  
stat -c '%U %G %a %n' /var/log/exam-audit.log
```

再生成新事件：

```
POST_ID="rhcsa13-postrotate-$(date +%s)"  
logger -p local0.notice -t exam-audit "$POST_ID"  
grep -F "$POST_ID" /var/log/exam-audit.log
```

验收矩阵：

1. 归档文件出现；
2. 新活动文件存在且身份正确；
3. 保留和压缩符合策略；
4. 新 POST_ID 进入新活动文件，而非旧归档。

⑦ [知识点] rename/create 后写入者可能仍持有旧 inode

logrotate 重命名文件并创建新文件，不会自动改变进程已经打开的文件描述符。若写入进程不 `reopen`，它会继续写入被重命名的归档。`postrotate` 的意义是通知进程重新打开路径。

`copytruncate` 在原路径复制后截断，可兼容无法 `reopen` 的应用，但复制和截断之间存在日志丢失窗口，因此不是默认首选。

⑧ [诊断] 配置正确却从未自动执行

应检查 logrotate 的调度入口和运行记录，例如 `logrotate.timer`、`logrotate.service` 及其 journal。timer 的日历和持久触发机制归第 15 章，本章只把它作为“策略从未被调用”的下一条证据。

⋮ {.cheatsheet} Cheatsheet 策略 + state + 文件决定轮转；`-d` 解释、不修改，`-f` 真操作；强制测试使用自包含片段和独立 state；最后用新 POST_ID 证明写入者切换。⋮ ⋮

⋮ {#RHCSA-13-D01 .topic .diagnosis data-kind="diagnosis-topic"}

[诊断专题] 日志查不到、写不到、转不到和轮不动

日志故障最容易陷入“多跑几次同一命令”。更有效的方法是把现象放回证据链，选择下一条最能区分假设的查询。每次修复后必须生成新事件，防止旧证据污染结论。

① [诊断] 已知事件在 `journalctl` 中查不到

症状：应用刚报错，但严格查询为空。假设：boot、时间、unit/tag、priority 或字段条件过窄；应用可能直接写文件。下一条证据：从宽到窄逐步移除条件。

```
journalctl --since '-30 min' --no-pager
journalctl -b --since '-30 min' --no-pager
journalctl -t <TAG> --since '-30 min' --no-pager
journalctl -u <UNIT> -b --since '-30 min' --no-pager
```

若宽查询仍无记录，再查应用自己的日志路径与输出方式。最小修复不是更换随机参数，而是修正错误范围或事件接入路径。

② [诊断] 只能看到当前 boot

```
--list-boots 只有 0
→ 检查有效 Storage=
→ 检查 /var/log/journal
→ 检查容量和 MaxRetentionSec
→ 检查是否执行过 vacuum
→ 检查读取权限
```

修改为持久存储只能保证后续 boot，无法恢复已经消失的上一 boot。再验证必须安排一次安全重启，并在 `-b -1` 中查找重启前 TEST_ID。

③ [诊断] journal 有 TEST_ID，目标文件没有

```
事件产生与 journald 收集基本成立
→ rsyslogd -N1
→ include 和 selector
→ rsyslog unit 日志
→ 目录/文件权限
→ AVC/SELinux
→ 最小修复
→ 新 TEST_ID
```

不要通过关闭 SELinux 或全局放宽权限跳过证据。若目标路径不是标准日志路径，需在 SELinux 章节进一步处理持久标签。

④ [诊断] 客户端已连接，接收端没有日志

TCP 连接只排除部分网络问题。继续检查：协议是否一致、接收端是否监听正确地址、input 是否加载、接收规则是否命中、action 是否有写入权限，最终在接收端寻找同一 TEST_ID。

⑤ [诊断] logrotate dry-run 表示“不需要轮转”

优先检查：

- 目标片段是否被 include
 - 文件是否匹配
 - 状态文件中的上次轮转时间
 - daily/weekly 与 size/minsize/maxsize 条件
 - 文件是否为空或缺失

若只是测试任务，使用独立 state 和自包含片段强制；若是生产对象，应尊重策略判断，不因“想看到 .1”而强制改变现场。

⑥ [诊断] 已经轮转，但新文件不增长

当前证据：归档存在、新活动文件存在、写入仍进入归档。假设：写入进程仍持有旧 inode。下一条证据：检查程序的 reopen 机制和打开文件，再核对 postrotate。最小修复：使用应用支持的 reload/HUP/reopen。再验证：新 POST_ID 只进入活动文件。

⑦ [安全边界] 磁盘压力与证据保全同时存在

日志占满磁盘时既要恢复可用性，也要保留调查证据。先识别增长最快的源和允许删除的范围；在无明确授权时，不删除全部 journal、不清空业务日志、不把长期预算问题掩盖为一次 vacuum。

∴ {.diagnosis-chain} 通用诊断链

- 症状
 - 当前证据属于哪一层
 - 列出 1~2 个可区分假设
 - 选择下一条最有区分度的证据
 - 做最小修复
 - 生成新的唯一事件
 - 在目标层再次验证

∴ ∴

∴ {#RHCSA-13-C01 .topic .classic .task-section data-kind="classic-task"}

[经典任务] 为指定 facility 建立本地日志路由和轮转

环境与当前状态

一台 RHEL 9 主机运行审计脚本，脚本通过以下形式写消息：

```
logger -p local0.notice -t exam-audit "<MESSAGE>"
```

当前系统存在其他 rsyslog 规则，不允许删除或覆盖。/var/log/exam-audit.log 尚未稳定接收新消息，也没有专用 logrotate 片段。

目标终态

- local0.notice 及更严重消息写入 /var/log/exam-audit.log ；
- 文件模式为 0640，所有者和组均为 root ；
- 日志按 daily 轮转，保留 7 份，旧归档压缩；
- 空文件不轮转，文件缺失不导致任务失败；
- 轮转后 rsyslog 继续向新的活动文件写入；
- 每层验收使用本次生成的唯一事件。

限制条件

- 不删除现有 rsyslog 规则；
- 不使用 chmod 777 ；
- 不关闭 SELinux；
- 不对整个 /etc/logrotate.conf 无调查执行 -f ；
- 不把 rsyslog.service active 当作最终结果；
- 未在目标主机完成真实轮转前，不得把配置核对写成“已验证成功”。

验收矩阵

层次	最少证据	不能扩大解释为
配置语法	rsyslogd -N1	selector 已命中
事件产生	journal 中出现 TEST_ID	文件已写入
本地路由	目标文件出现同一 TEST_ID	轮转策略正确
文件身份	stat 显示 root root 640	写入者已 reopen
轮转决策	logrotate -d 解释目标策略	已真实轮转
真实轮转	受控 -s ... -f 后出现归档	后续仍写新文件

层次	最少证据	不能扩大解释为
写入者切换	新 POST_ID 进入活动文件	所有生产场景均验证
...		

... {#RHCSA-13-C02 .topic .classic .task-section data-kind="classic-task"}

[经典任务] 调查只能看到当前 boot、看不到上一次启动日志

环境与当前状态

服务器刚经历一次非计划重启。管理员执行：

```
journalctl --list-boots
journalctl -b -1
```

只能看到当前 boot。业务窗口暂不允许再次重启。

目标终态

- 区分“未启用持久 journal”“旧记录已被策略清理”“当前用户权限不足”；
- 使用 `/etc/systemd/journald.conf.d/60-persistent.conf` 明确持久存储；
- 将持久 journal 最大使用量限制为 `512M`，最长保留 `30day`；
- 生成唯一验证事件并完成当前可验证层次；
- 写出下一次安全重启后的跨 boot 验收命令；
- 明确此前已经丢失的历史不能被新配置恢复。

限制条件

- 不删除 `/var/log/journal`；
- 调查期间不执行 `vacuum`；
- 不以当前 boot 的测试事件声称跨 boot 已成功；
- 不抢先展开 `chrony` 调优；
- 当前窗口不允许重启时，必须将跨 boot 验收明确标为待执行。

验收矩阵

层次	当前可完成	需要重启后完成
有效配置	检查 <code>Storage=</code> 和 <code>drop-in</code>	确认配置持续有效
存储对象	检查 <code>/var/log/journal</code> 与占用	确认上一个 boot 仍在列表

层次	当前可完成	需要重启后完成
当前事件	当前 boot 中找到 TEST_ID	<code>-b -1</code> 找到同一 TEST_ID
保留预算	静态核对 512M/30day	长期观察是否符合预算
历史恢复	明确不可恢复边界	不适用
...		

```
::: {#RHCSA-13-A01 .topic .answer .answer-section data-kind="answer-topic"}
```

[参考解答] 任务一：本地路由、轮转与写入者重新打开

① 调查当前对象和冲突

```
grep -R -n 'local0\|exam-audit' \
    /etc/rsyslog.conf /etc/rsyslog.d 2>/dev/null
rsyslogd -N1
ls -l /var/log/exam-audit.log* 2>/dev/null
```

先确认是否已有 local0 规则和同名文件。若已有规则允许多目标写入，则保留；若规则冲突，先解释影响再做最小修改。

② 写入最小 rsyslog 规则

```
# /etc/rsyslog.d/60-exam-audit.conf
local0.notice    /var/log/exam-audit.log
```

选择 `local0.notice`，因为任务要求 notice 以及更严重消息。`local0.=notice` 会漏掉 warning、err 等更严重级别。

```
rsyslogd -N1
systemctl restart rsyslog.service
systemctl is-active rsyslog.service
journalctl -u rsyslog.service -b -n 30 --no-pager
```

`-N1` 是配置语法证据；`restart` 和 `active` 是加载与运行层证据，仍不是路由结果。

③ 生成唯一事件并完成本地路由验收

```
TEST_ID="rhcsa13-local-$(date +%s)"
logger -p local0.notice -t exam-audit "$TEST_ID"

journalctl -t exam-audit --since '-2 min' --no-pager
grep -F "$TEST_ID" /var/log/exam-audit.log
stat -c '%U %G %a %n' /var/log/exam-audit.log
```

若 journal 有而文件无，按“配置解析 → include-selector → unit 日志 → 路径权限 → AVC”推进。修复后创建新的 TEST_ID，不能用旧消息验收新配置。

④ 写入 logrotate 策略

```
# /etc/logrotate.d/exam-audit
/var/log/exam-audit.log {
    daily
    rotate 7
    compress
    missingok
    notifempty
    create 0640 root root
    postrotate
        /usr/bin/systemctl kill -s HUP --kill-whom=main \
            rsyslog.service >/dev/null 2>&1 || true
    endscrip
}
```

参数选择：

- `daily`：每天进入一次时间判断；
- `rotate 7`：保留 7 份；
- `compress`：压缩旧归档；
- `missingok`：文件缺失时不报致命错误；
- `notifempty`：空文件不轮转；
- `create 0640 root root`：创建新活动文件；
- `postrotate`：请求 rsyslog 重新打开路径。

真实主机应核对系统自带 rsyslog 片段的 reopen 方式；不同版本或应用不能机械复制同一 postrotate。

⑤ 先解释轮转决策

```
logrotate -d /etc/logrotate.conf
```

确认目标片段被 include、路径被匹配、状态文件被读取，并记录本次是否到期及原因。debug 不改变文件。

⑥ 在受控状态下执行真实轮转

确认片段只包含测试目标后：

```
logrotate -s /tmp/rhcsa13-exam-audit.status \  
-f /etc/logrotate.d/exam-audit  
  
ls -l /var/log/exam-audit.log*  
stat -c '%U %G %a %n' /var/log/exam-audit.log
```

-s 隔离状态文件，-f 仍会真实改变目标日志，因此只能在明确任务对象上执行。

⑦ 用新 POST_ID 证明写入者切换

```
POST_ID="rhcsa13-postrotate-$(date +%s)"  
logger -p local0.notice -t exam-audit "$POST_ID"  
  
grep -F "$POST_ID" /var/log/exam-audit.log  
grep -R -F "$POST_ID" /var/log/exam-audit.log.* 2>/dev/null
```

期望：POST_ID 出现在新的活动文件，不出现在旧归档。只看到 .1 或 .gz 不能证明写入者已 reopen。

典型错误与分支

- local0.=notice：范围过窄；改为阈值 selector。
- rsyslogd -N1 通过但文件无消息：检查加载、selector、路径和 AVC。
- logrotate -d 表示未到期：若是考试受控测试，使用独立 state 强制；生产场景尊重策略。
- 轮转后仍写归档：修复 reopen，而不是立刻改用 copytruncate。
- copytruncate：只在应用无法 reopen 时评估，并接受复制/截断窗口的潜在丢失风险。

验证边界

rsyslogd -N1、配置文本和 dry-run 只能证明静态层次。真实轮转、权限保持以及写入者重新打开必须在目标 RHEL 9 主机上完成：postrotate 的信号或命令还应与系统自带 rsyslog 片段核对。 :::

```
::: {#RHCSA-13-A02 .topic .answer .answer-section data-kind="answer-topic"}
```

[参考解答] 任务二：持久 journal 与跨 boot 验证计划

① 调查当前 boot、占用与配置来源

```
journalctl --list-boots  
journalctl --disk-usage  
ls -ld /run/log/journal /var/log/journal 2>/dev/null  
systemd-analyze cat-config systemd/journald.conf
```

当前只有一个 boot 时，保留三类假设：从未持久、旧记录已清理、权限不足。若 `cat-config` 不可用，再逐个读取主配置和 drop-in，不能只看一个文件就认定有效值。

② 建立管理员配置目录并保留已有文件

```
mkdir -p /etc/systemd/journald.conf.d
cp -a /etc/systemd/journald.conf.d/60-persistent.conf \
    /etc/systemd/journald.conf.d/60-persistent.conf.bak \
    2>/dev/null || true
```

只备份存在的管理员文件，不修改 `/usr/lib` 中的 vendor 配置。

③ 写入明确的持久与保留策略

```
# /etc/systemd/journald.conf.d/60-persistent.conf
[Journal]
Storage=persistent
SystemMaxUse=512M
MaxRetentionSec=30day
```

512M 和 30day 来自任务目标。它们表达上限与最长保留，不保证每条日志一定保留满 30 天；磁盘压力和容量上限可能更早触发轮转与清理。

④ 让运行中的 journald 读取配置

```
systemctl restart systemd-journald.service
journalctl --flush
journalctl --disk-usage
```

这一步完成配置应用与当前占用观察。它不能证明跨重启，因为重启尚未发生。

⑤ 建立重启前唯一证据

```
TEST_ID="rhcsa13-persist-$(date +%s)"
logger -p local0.notice -t rhcsa13-persist "$TEST_ID"

journalctl -t rhcsa13-persist --since '-2 min' --no-pager
journalctl --list-boots
```

把 TEST_ID 记录到操作草稿。当前查询证明事件已进入当前 journal，不证明它会出现在下一次 `-b -1`。

⑥ 在下次安全重启后完成跨 boot 验收

```
journalctl --list-boots
journalctl -b -1 -t rhcsa13-persist --no-pager
```

只有列表出现上一 boot，且重启前 TEST_ID 能在 `-b -1` 中找到，才能写出“持久 journal 跨重启验证成功”。若尚未实施安全重启，该项必须保持为待验证。

⑦ 解释无法恢复的边界

若历史曾只保存在 `/run/log/journal` 并随重启消失，或已经被容量/vacuum 清理，新配置无法恢复。正确结论是“已为后续 boot 建立持久化和保留预算”，不是“恢复了上一次启动日志”。

典型错误与分支

- 只创建 `/var/log/journal`，不检查有效 `Storage=`；
- 修改配置后不让 `journald` 读取；
- 当前 TEST_ID 可见就声称跨 boot 成功；
- 调查期间执行 `vacuum`，破坏剩余证据；
- `--since` 查询为空便立即调整 `chrony`，而未先放宽范围；
- 把 `SystemMaxUse=512M` 理解为磁盘会立即占满 512M。

验证边界

配置解析、当前事件与磁盘占用属于重启前证据；目录创建、flush 后落盘、跨重启保留和长期容量效果仍需在目标 RHEL 9 主机上分层验证。 ∴

∴ {#RHCSA-13-S01 .topic .wrap data-kind="chapter-wrap"}

[本章收束] 从“看到日志”迁移到“维护证据链”

日志能力的核心不是记住更多路径，而是把每条结论落到明确对象和证据层。稳定工作方法是：先确认事件来源和查询范围，再用结构化字段缩小对象；配置路由前做静态检查，配置后用新 TEST_ID 完成目标层验收；清理和强制轮转前先解释状态与风险。

工作方法：七步闭环

1. 明确事件来源与目标证据位置
2. 先定 boot 和时间，再定 unit/tag/字段
3. 用 verbose 发现字段，用最小条件缩小对象
4. 变更配置前建立基线并执行静态检查
5. 生成新的唯一测试事件
6. 在 journal、文件或接收端逐层验证
7. 对持久、保留和轮转后写入做独立验收

主要判断表

看到的证据	可以证明	不能证明	下一层证据
<code>journalctl -u ...</code> 有记录	当前 journal 有 unit 相关 entry	服务功能正确	监听、访问或业务检查
<code>--list-boots</code> 只有 0	当前可用 journal 只有当前 boot	上一 boot 没事件	有效 Storage、目录、保留与权限
<code>rsyslogd -N1</code> 通过	配置可解析	selector 命中	新 TEST_ID 进入目标文件
rsyslog active	进程当前运行	新规则已加载且有效	unit 日志与目标文件 TEST_ID
TCP 514 已连接	传输层连接建立	接收端已落盘	接收端同一 TEST_ID
<code>logrotate -d</code> 显示策略	配置被读取并解释决策	文件已经轮转	受控真实轮转与文件检查
出现 <code>.1</code> / <code>.gz</code>	产生了归档	写入者切到新文件	新 POST_ID 进入活动文件
当前 TEST_ID 可查	当前 entry 存在	跨重启持久	重启后 <code>-b -1</code> 同一 TEST_ID
<code>--disk-usage</code> 显示占用	当前 journal 体积	保留政策合理	日志速率、预算与审计要求

考试与工作迁移

考试题通常给出明确 facility、文件路径、保留数量或持久化参数；真实工作还要考虑集中平台、TLS、队列、审计、脱敏和容量预测。无论规模如何扩大，底层方法仍是“对象明确、范围明确、证据分层、变更最小、验证不可扩大”。

向下一章交接

本章在查询时间窗口和跨主机排序时，只把“当前时间、时区与同步状态”当作一条独立证据。下一章《系统时间、时区、RTC 与 chrony》将继续回答：日志时间戳为何偏移、主机时钟如何建立可信来源、RTC 与系统时钟如何交互，以及怎样验证时间同步终态。 ∴